



AI-Powered Interview Simulator Using Local LLM Models&RAG

A PROJECT REPORT

Submitted by

DAKSHIN ADITYA V	211722118013
DEEPANSHU MOHANTY	211722118014
KARTHICK RAJA M K	211722118030

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER AND COMMUNICATION ENGINEERING

RAJALAKSHMI INSTITUTE OF TECHNOLOGY, KUTHAMBAKKAM

ANNA UNIVERSITY: CHENNAI 600 025

APRIL 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

College Name : 2117- Rajalakshmi Institute of Technology
Branch & Semester : Computer and Communication Engineering & VIII Semester
Subject Code & Name : CO3811 / Project Work

Certified that this project report titled "**ADAPTIVE INTERVIEW EVALUATION USING LLM AND RETRIEVAL-AUGMENTED GENERATION**" is the bonafide work of **V DAKSHIN ADITYA [211722118013], DEEPANSHU MOHANTY [211722118014] and KARTHICK RAJA M K [211722118030]** , who carried out the project work under my guidance. Further certified that, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation based on which a degree or award was conferred on an earlier occasion to any other candidate.

SUPERVISOR

Dr. E.GANESH,M.E., Ph.D.,
Assistant Professor
Computer and Communication Engineering
Rajalakshmi Institute of Technology
Chennai-600 124

HEAD OF THE DEPARTMENT

Dr. E.GANESH,M.E., Ph.D.,
Assistant Professor & Head
Computer and Communication Engineering
Rajalakshmi Institute of Technology
Chennai-600 124

The Project Viva- voce held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We wish to express our heartly and sincere gratitude to our chairman **Mr.S.MEGANATHAN**, B.E.,F.I.E., for his sincere endeavor in educating using his premier institution.

We wish to express our deep gratitude to our chairperson **Dr. (Mrs.) THANGAM MEGANATHAN**, M.A., M.Phil., Ph.D., for enthusiastic motivation which helped us a lot in completing the project.

We wish to express our sincere gratitude to our vice chairman **Dr.HAREE SHANKAR MEGANATHAN**, MBBS.,MD., for his constant motivation in educating us in his premier institution.

We express our thankfulness to **Dr. R. MAHESWARI**, M.E, Ph.D., Principal, and **Dr. R. SUNDAR**, M.Tech, Ph.D., Director, Rajalakshmi Institute of Technology for their kind support and the facilities provided to complete our work on time.

With profound sense and regards, we acknowledge with great pleasure **Dr. E. GANESH**, M.E., Ph.D., Assistant Professor & Head, Department of Electronics and Communication Engineering for her valuable suggestions and guidance for the development and completion of our project.

We express our sincere thanks to our supervisor **Dr. E. GANESH**, M.E., Ph.D., Assistant Professor & Head ,Department of Computer and Communication Engineering, for leading us on the project.

We extend our gratitude to **Mr. S. BHARATH**, M.E., (Ph.D.), Assistant Professor, Project Coordinator for their timely organization of reviews and their support throughout the project work.

Finally, we express our deep sense of gratitude to our parents, all our faculty members, technical staff and all our friends for their constant encouragement and moral support.

ABSTRACT

Candidates who prepare for interviews gain confidence, increasing their chances of getting hired. Current mock interview systems rely on fixed question banks, rule-based evaluation methods, and cloud platforms that create security risks, increase operational costs, and limit personalized assessments. This paper presents an artificial intelligence interview simulation system that uses local Large Language Models (LLMs) and a Retrieval-Augmented Generation (RAG) framework to provide secure and personalized interview training. The system processes PDF and DOCX resumes submitted by users to extract relevant skills and competencies using spaCy and a custom skill dictionary. Locally deployed LLMs generate domain-specific interview questions based on extracted candidate profiles. The RAG framework, using FAISS and MiniLM embeddings, provides contextual grounding and improves factual consistency in question generation and response evaluation. User responses are assessed through a structured grading system that ensures fair and consistent evaluation. The system runs on Ollama-based LLMs entirely on local machines, ensuring data privacy, reducing operational costs, and enabling offline access. The intelligent interview training platform demonstrates effectiveness by generating adaptive questions, providing accurate assessments, and tracking user progress across sessions while enabling real-time system interaction.

TABLE OF CONTENTS

CHAPTER NO	TOPIC	PAGE NO
	ABSTRACT	III
	LIST OF FIGURES	XI
	LIST OF SYMBOLS	XII
	LIST OF ABBREVIATIONS	XIII
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 NEED FOR INTELLIGENT INTERVIEW PREPARATION SYSTEMS	2
	1.3 PROBLEM STATEMENT	4
	1.4 PROPOSED SYSTEM	5
	1.5 OBJECTIVES OF THE PROJECT	7
	1.6 SCOPE OF THE PROJECT	7
2	LITERATURE SURVEY	8
	2.1 INTELLIGENT INTERVIEW SYSTEMS USING LARGE LANGUAGE MODELS	8
	2.2 RESUME PARSING AND SKILL EXTRACTION USING NLP	9

	2.3 LARGE LANGUAGE MODELS FOR AI- BASED QUESTION GENERATION	10
	2.4 RETRIEVAL-AUGMENTED GENERATION FOR KNOWLEDGE GROUNDING	11
	2.5 AUTOMATED CANDIDATE EVALUATION USING LARGE LANGUAGE MODELS	12
	2.6 PRIVACY-AWARE AI DEPLOYMENT IN RECRUITMENT SYSTEMS	13
	2.7 RESEARCH GAP AND MOTIVATION FOR PROPOSED SYSTEM	13
3	DATASET DESCRIPTION	15
	3.1 OVERVIEW OF THE RESUME DATASET	15
	3.2 RESUME ATTRIBUTES AND FEATURE CATEGORIES	16
	3.3 SKILL EXTRACTION AND TEXTUAL FEATURE REPRESENTATION	16
	3.4 FEATURE CORRELATION AND SIMILARITY ANALYSIS	17

	3.5 DATA QUALITY AND PREPROCESSING CONSIDERATIONS	18
	3.6 CANDIDATE DIVERSITY AND RANKING DISTRIBUTION	18
	3.7 RELEVANCE TO THE PROPOSED SYSTEM	19
4	SYSTEM ARCHITECTURE	20
	4.1 OVERVIEW OF THE PROPOSED SYSTEM	20
	4.2 FASTAPI CONTROLLER LAYER	21
	4.3 CORE PROCESSING PIPELINE	22
	4.4 RETRIEVAL-AUGMENTED GENERATION FRAMEWORK	23
	4.5 LOCAL LLM-BASED QUESTION GENERATION	23
	4.6 ANSWER EVALUATION AND SCORING MODULE	24
	4.7 SCORING AND SESSION MANAGEMENT	24
	4.8 DATABASE ADAPTER LAYER	24
	4.9 END-TO-END SYSTEM WORKFLOW	25

	4.10 SUMMARY	25
5	METHODOLOGY	26
	5.1 OBJECTIVES OF THE PROPOSED METHODOLOGY	26
	5.2 OVERALL METHODOLOGICAL WORKFLOW	27
	5.3 RESUME PARSING AND PREPROCESSING	27
	5.4 SKILL EXTRACTION AND FEATURE ENGINEERING	28
	5.5 SIMILARITY-BASED RANKING MECHANISM	28
	5.6 RETRIEVAL-AUGMENTED GENERATION (RAG) FRAMEWORK	29
	5.7 LOCAL LLM-BASED QUESTION GENERATION	29
	5.8 ANSWER EVALUATION AND RUBRIC- BASED SCORING	30

5.9 PERFORMANCE ANALYTICS AND PROGRESS TRACKING	30
5.10 PRIVACY AND DEPLOYMENT CONSIDERATIONS	31
5.11 CHAPTER SUMMARY	31
6 IMPLEMENTATION	32
6.1 SOFTWARE AND DEVELOPMENT ENVIRONMENT	32
6.2 DATASET INTEGRATION AND STRUCTURING	33
6.3 IMPLEMENTATION OF RESUME PREPROCESSING TECHNIQUES	34
6.3.1 Handling heterogeneous resume formats	34
6.3.2 Skill normalization and feature standardization	34
6.4 EXPLORATORY ANALYSIS INTEGRATION IN IMPLEMENTATION	35

6.5 IMPLEMENTATION OF SIMILARITY- BASED RANKING MECHANISM	35
6.6 IMPLEMENTATION OF RETRIEVAL- AUGMENTED GENERATION FRAMEWORK	36
6.7 IMPLEMENTATION OF LOCAL LLM-BASED QUESTION GENERATION	36
6.8 IMPLEMENTATION OF ANSWER EVALUATION AND SCORING	37
6.9 REAL-TIME USER INTERACTION INTERFACE	38
6.10 END-TO-END EXECUTION PIPELINE	38
6.11 AUTOMATED FEEDBACK REPORT GENERATION IMPLEMENTATION	39
6.12 SCALABILITY, DEPLOYMENT, AND ETHICAL CONSIDERATIONS	39
7 EXPERIMENTAL RESULTS	41
7.1 EXPERIMENTAL SETUP AND EVALUATION PROTOCOL	41

	7.2 SKILL DETECTION PERFORMANCE	42
	ACROSS LANGUAGE MODELS	
	7.3 COMPARATIVE ANALYSIS OF MODEL	44
	CLASSIFICATION ACCURACY	
	7.4 EVALUATION OF QUESTION	45
	GENERATION QUALITY	
	7.5 REAL-TIME RANKING AND INTERVIEW	46
	SIMULATION	
	7.6 AUTOMATED ANSWER EVALUATION AND	47
	FEEDBACK ANALYSIS	
	7.7 DISCUSSION ON PRACTICAL	48
	SIGNIFICANCE	
	7.8 LIMITATIONS OF EXPERIMENTAL	48
	EVALUATION	
	7.9 SUMMARY OF EXPERIMENTAL FINDINGS	49
8	CONCLUSION	50
9	FUTURE SCOPE	51
	REFERENCES	52

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
4.1	System Architecture	21
7.1	Heatmap of skill detection by different models	43
7.2	Comparative analysis of models classification accuracy	45
7.3	Comparative analysis of models question quality	46

LIST OF SYMBOLS

SYMBOL	DESCRIPTION
(x)	Input resume document
(y)	Predicted ranking score
$(f(x))$	Ranking function applied to resume data
(d)	Job description document
(S)	Similarity score
$\cos(\theta)$	Cosine similarity between resume and job description vectors
(v_r)	Resume embedding vector
(v_j)	Job description embedding vector
(TF)	Term Frequency in TF-IDF
(IDF)	Inverse Document Frequency
(E)	Embedding representation of text
(k)	Number of retrieved documents in RAG
(Q)	Generated interview question
(A)	Candidate answer
$(Score)$	Evaluation score assigned to candidate response
(Acc)	Accuracy metric
$(Prec)$	Precision metric
(Rec)	Recall metric
$(F1)$	F1-score evaluation metric

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
ATS	Applicant Tracking System
TF-IDF	Term Frequency – Inverse Document Frequency
API	Application Programming Interface
FAISS	Facebook AI Similarity Search
NER	Named Entity Recognition
PDF	Portable Document Format
DOCX	Microsoft Word Document Format
DB	Database
SQL	Structured Query Language
BLEU	Bilingual Evaluation Understudy
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
EDA	Exploratory Data Analysis
XAI	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

In today's highly competitive job market, securing employment requires more than just academic qualifications and technical knowledge. Organizations increasingly focus on evaluating candidates based on their ability to communicate ideas clearly, demonstrate problem-solving abilities, and apply their knowledge in real-world situations. As a result, the interview process has become one of the most critical stages in the recruitment process.

Interviews provide employers with an opportunity to assess a candidate's technical expertise, analytical thinking, and communication skills. For job seekers, performing well in interviews plays a vital role in determining whether they will be selected for a particular role. Even candidates with strong academic backgrounds may struggle to perform effectively during interviews if they lack proper preparation or confidence. Traditionally, candidates prepare for interviews through self-study, attending coaching programs, reading interview preparation books, or practicing with fixed question banks available on the internet. Some candidates also participate in mock interviews with peers or mentors to gain practical experience. Although these approaches provide basic preparation, they have several limitations. Fixed question banks often contain generic questions that are not aligned with the specific skills or experiences of individual candidates. Peer mock interviews may vary in quality and often lack objective evaluation criteria.

In addition, traditional interview preparation methods do not provide structured feedback that helps candidates understand their weaknesses or

improve their responses. Without proper evaluation and feedback, candidates may not be able to identify areas that require improvement.

Another challenge arises in the recruitment process itself. Organizations receive a large number of job applications for each open position. Manually reviewing every resume becomes time-consuming and inefficient for recruiters. Many companies therefore use automated tools such as Applicant Tracking Systems (ATS) to filter resumes based on keyword matching. However, these systems often fail to capture the semantic meaning of candidate skills and experiences, leading to inaccurate candidate screening. These challenges highlight the need for intelligent systems that can automate resume analysis, generate personalized interview questions, and evaluate candidate responses effectively.

1.2 NEED FOR INTELLIGENT INTERVIEW PREPARATION SYSTEMS

With the rapid advancement of technology, Artificial Intelligence (AI) and Natural Language Processing (NLP) have emerged as powerful tools for developing intelligent systems capable of understanding and generating human language. These technologies have significantly improved applications such as chatbots, automated translation, sentiment analysis, and text summarization.

In recent years, Large Language Models (LLMs) have gained significant attention due to their ability to understand complex language patterns and generate coherent responses. These models are trained on massive datasets and can perform tasks such as answering questions, summarizing information,

generating text, and evaluating responses. In the context of recruitment and interview preparation, LLMs can be used to simulate real interview environments. They can generate interview questions based on a candidate's skills and experience, evaluate candidate responses, and provide meaningful feedback. This capability allows candidates to practice interviews in a realistic and interactive environment.

Another important development is the use of Retrieval-Augmented Generation (RAG). RAG enhances language model performance by retrieving relevant information from external knowledge sources before generating responses. By combining retrieval mechanisms with language generation models, RAG improves factual accuracy and ensures that generated content is grounded in reliable information. Despite these technological advancements, many existing AI-based interview preparation platforms rely heavily on cloud-based services. These systems require continuous internet connectivity and involve sending sensitive user data, such as resumes and interview responses, to external servers. This raises significant concerns regarding data privacy and security.

Additionally, cloud-based platforms often require paid subscriptions or usage fees, making them less accessible for students and educational institutions with limited financial resources. In environments where reliable internet connectivity is unavailable, the usability of such systems becomes even more limited. Another limitation of existing platforms is the lack of personalization. Many systems generate generic interview questions that do not reflect the candidate's specific skills, projects, or career goals. As a result, candidates do not receive a realistic interview preparation experience tailored to their individual profiles. Therefore, there is a growing need for an

intelligent interview preparation system that can operate locally, protect user data, and generate personalized interview questions based on resume content.

1.3 PROBLEM STATEMENT

Although several tools and platforms have been developed to support recruitment and interview preparation, many of them still suffer from significant limitations. One of the primary challenges lies in resume analysis. Most automated resume screening systems rely on keyword-based filtering techniques. These systems identify candidates by matching keywords from resumes with keywords from job descriptions. However, this approach does not capture the semantic meaning of candidate skills and experiences. Candidates may possess the required skills but use different terminology in their resumes, leading to inaccurate screening results.

Another major limitation is the lack of personalized interview preparation. Many existing platforms provide a fixed set of interview questions that are not tailored to individual candidates. This prevents candidates from practicing questions related to their specific technical skills or projects. Automated evaluation of interview responses also remains a difficult problem. Traditional rule-based evaluation systems depend on simple keyword matching and predefined scoring patterns. Such systems cannot interpret the conceptual understanding, reasoning ability, or communication clarity demonstrated in open-ended responses.

Furthermore, cloud-based interview preparation platforms raise significant privacy concerns. Candidate resumes often contain sensitive information such as educational background, work experience, project descriptions, and

personal contact details. Transmitting this information to external servers increases the risk of data breaches or misuse. Another challenge is the absence of structured performance tracking in many existing systems. Candidates do not receive detailed insights into their performance over time, making it difficult for them to monitor improvement or identify specific areas that require additional practice. These challenges indicate the need for an intelligent system that can analyze resumes accurately, generate personalized interview questions, evaluate responses effectively, and provide structured feedback while ensuring data privacy.

1.4 PROPOSED SYSTEM

To address the limitations of existing systems, this project proposes a Smart Resume Ranker and Adaptive Interview Evaluation System that integrates advanced machine learning and natural language processing techniques. The proposed system is designed to provide an intelligent interview preparation platform that analyzes candidate resumes, generates personalized interview questions, evaluates responses, and provides feedback for improvement.

The first step in the system is resume parsing and preprocessing. The system accepts resumes in commonly used formats such as PDF and DOCX. Text extraction tools are used to convert the documents into structured text for analysis. Next, the system performs skill extraction using the spaCy NLP framework. spaCy uses Named Entity Recognition and linguistic analysis to identify technical and soft skills mentioned in the resume. A custom skill dictionary is used to improve the accuracy of skill detection across different technical domains. After extracting the relevant skills, the system converts the

textual information into numerical representations using MiniLM sentence embeddings. These embeddings capture semantic relationships between words and phrases, allowing the system to understand the contextual meaning of resume content.

The system then performs similarity-based candidate ranking by comparing resume embeddings with job description embeddings using cosine similarity. This approach allows the system to rank candidates based on contextual relevance rather than simple keyword matching. To generate personalized interview questions, the system integrates a Retrieval-Augmented Generation (RAG) framework. The RAG architecture retrieves relevant information from a curated knowledge base and provides it as contextual input to the language model before generating questions.

The question generation process is powered by locally deployed Large Language Models using Ollama. By running the language model locally, the system ensures that all resume data and user responses remain on the user's device. This eliminates the need for cloud services and ensures complete data privacy. In addition to generating interview questions, the system includes an automated answer evaluation module. This module evaluates candidate responses using a structured rubric that considers factors such as technical accuracy, completeness, relevance, and clarity of explanation.

1.5 OBJECTIVES OF THE PROJECT

The main objectives of this project are:

1. To develop an intelligent system for automated resume analysis and skill extraction.

2. To implement similarity-based candidate ranking using semantic embeddings.
3. To generate personalized interview questions based on candidate skills.
4. To evaluate candidate responses using an automated rubric-based scoring system.
5. To provide structured feedback and performance tracking for interview preparation.
6. To ensure data privacy by deploying Large Language Models locally.

1.6 SCOPE OF THE PROJECT

The scope of this project focuses on developing an intelligent interview preparation system that can assist both candidates and recruiters. The system is designed to analyze technical resumes, generate personalized interview questions, and evaluate responses using AI-based methods. The project primarily targets candidates in technical domains such as Computer Science, Information Technology, Artificial Intelligence, and Data Science. However, the architecture can be extended to support other domains with appropriate knowledge bases and skill dictionaries. Future improvements may include integration with web-based platforms, support for multiple job roles, voice-based interview simulation, and advanced analytics dashboards for recruiters and candidates.

CHAPTER 2

LITERATURE SURVEY

Artificial Intelligence systems developed using Natural Language Processing (NLP) technologies have significantly transformed recruitment and interview preparation platforms. Modern AI-driven systems can analyze resumes, generate interview questions, perform automated evaluation of responses, and deliver personalized training experiences for candidates. Recent research in this field has explored several important areas including Large Language Model (LLM) based evaluation systems, resume parsing and skill extraction, AI-driven question generation, Retrieval-Augmented Generation (RAG) architectures, speech and sentiment analysis, and automated interview assessment frameworks. The following sections review important research contributions related to these domains.

2.1 INTELLIGENT INTERVIEW SYSTEMS USING LARGE LANGUAGE MODELS

Large Language Models have emerged as powerful tools for building intelligent interview simulation systems. Their ability to understand context and generate human-like responses enables them to create realistic interview experiences for candidates.

Xinyue et al. (2024) introduced an interview simulation system based on Large Language Models combined with multi-agent interactions ^[1]. In this system, multiple AI agents collaborate to generate and refine interview questions. Each agent evaluates the quality of generated questions and provides feedback to other agents in order to improve clarity, contextual

relevance, and accuracy. The collaborative process results in high-quality interview questions that closely resemble real-world interview scenarios. The study demonstrates that LLM-driven interview systems can significantly enhance personalized interview preparation by creating adaptive and interactive learning environments. This research highlights the potential of multi-agent AI architectures in improving interview training platforms.

2.2 RESUME PARSING AND SKILL EXTRACTION USING NLP

Resume parsing is one of the most critical components in intelligent recruitment systems. Extracting relevant information such as skills, education, experience, and certifications from unstructured resumes allows AI systems to build accurate candidate profiles.

Tallapragada et al. (2023) proposed an improved resume parsing framework that uses BERT-based contextual embeddings to extract meaningful information from resumes ^[2]. Unlike traditional keyword-matching approaches, the BERT-based system captures semantic relationships between skills, job roles, and experiences. This allows the model to better understand resume content and significantly improves extraction accuracy.

Similarly, Abisha et al. (2024) introduced Resspar, an AI-driven resume parsing and recruitment system that combines Natural Language Processing with generative AI techniques ^[3]. The system processes both structured and unstructured resume formats, including scanned documents. It extracts candidate attributes and converts them into structured data for further analysis. The framework improves reliability in resume comprehension and enables automated candidate screening.

Another study conducted by Chandak et al. (2024) developed a resume parser and job recommendation system using machine learning techniques [7]. Their system integrates spaCy-based NLP pipelines with PDF processing tools to extract candidate skills and experiences from resumes. The extracted data is then used to recommend suitable job roles for candidates. These studies demonstrate that advanced NLP techniques significantly improve resume parsing and candidate profiling, enabling personalized interview preparation systems.

2.3 LARGE LANGUAGE MODELS FOR AI-BASED QUESTION GENERATION

Automatic question generation is another important research area in AI-based interview systems. Generating relevant and diverse questions requires deep contextual understanding of candidate skills and domain knowledge.

Hang et al. (2024) introduced MCQGen, a Large Language Model-driven question generation system designed for personalized learning environments^[4]. The system uses Retrieval-Augmented Generation and prompt engineering techniques to generate high-quality multiple-choice questions. The use of knowledge retrieval ensures that generated questions remain factually accurate and contextually relevant.

A systematic literature review conducted by Awalurahman et al. (2025) examined transformer-based automatic question generation techniques^[5]. The study analyzed various methods such as fine-tuning, zero-shot prompting, few-shot learning, and chain-of-thought reasoning. The results indicate that LLM-based question generation systems outperform traditional rule-based

methods in terms of coherence, diversity, and contextual relevance. Similarly, Maity et al. (2025) evaluated the ability of Large Language Models to generate educational-level questions ^[15]. Their research measured question quality based on relevance, grammatical correctness, usefulness, and answerability. The results show that LLMs can generate high-quality questions when guided by appropriate constraints and contextual information. These studies confirm that LLMs are effective tools for generating intelligent interview questions.

2.4 RETRIEVAL-AUGMENTED GENERATION FOR KNOWLEDGE GROUNDING

Retrieval-Augmented Generation has emerged as a powerful architecture for improving the reliability of Large Language Models. RAG systems combine information retrieval mechanisms with generative models to produce knowledge-grounded responses.

Gao et al. (2023) provided a comprehensive survey on Retrieval-Augmented Generation architectures ^[8]. The study categorizes RAG systems into basic, modular, and hybrid frameworks and explains how retrieval mechanisms improve factual consistency and reduce hallucination in language models.

Patel et al. (2024) conducted a comparative study on Large Language Models integrated with RAG-based question answering systems ^[6]. Their research shows that retrieval-based grounding significantly improves contextual accuracy and response stability.

Further research by Sharma et al. (2025) analyzed advanced RAG architectures and evaluated their robustness and scalability in knowledge-grounded generation systems ^[11]. Their study highlights that hybrid RAG frameworks provide consistent output and improved reasoning capabilities.

Singh et al. (2025) introduced Agentic Retrieval-Augmented Generation, where autonomous agents manage retrieval, reasoning, and generation tasks ^[12]. The system improves response quality by allowing agents to plan and refine information retrieval processes. Peng et al. (2024) proposed Graph Retrieval-Augmented Generation, which integrates graph-based knowledge representations into RAG systems ^[13]. This approach enhances semantic understanding by capturing relationships between different concepts in the knowledge base. These studies demonstrate that RAG architectures play a crucial role in improving contextual understanding and knowledge grounding in AI systems.

2.5 AUTOMATED CANDIDATE EVALUATION USING LARGE LANGUAGE MODELS

Automated evaluation of interview responses remains a challenging problem in AI-based recruitment systems. Traditional rule-based evaluation systems rely on keyword matching and predefined scoring patterns, which often fail to capture semantic meaning. Xie et al. (2024) proposed a rubric-based automated assessment system using Large Language Models ^[9]. Their framework evaluates responses based on structured grading criteria and demonstrates strong alignment with human evaluation standards.

Chu et al. (2024) introduced an LLM-powered automatic grading framework that uses multi-agent collaboration to improve grading accuracy^[10]. The system employs reflective agents and optimization agents to refine grading rules and enhance evaluation reliability. Gao et al. (2024) conducted a systematic review of automated text-based assessment systems in higher education^[14]. Their findings indicate that rubric-guided LLM systems produce the most consistent and interpretable evaluation results. These studies highlight that structured LLM-based evaluation frameworks can effectively replicate human-like grading for open-ended responses.

2.6 PRIVACY-AWARE AI DEPLOYMENT IN RECRUITMENT SYSTEMS

Many AI-based recruitment platforms rely on cloud infrastructure to process candidate data. While cloud computing provides scalability, it also introduces security and privacy concerns. Sensitive information such as candidate resumes, work experience, and personal details must often be transmitted to external servers when using cloud-based AI systems. This increases the risk of data breaches and unauthorized access. Recent research emphasizes the importance of local deployment of AI models to ensure data privacy and offline accessibility. Hybrid RAG architectures have demonstrated that AI systems can function effectively in local environments without relying on external cloud services^[11]. Local deployment frameworks enable organizations and educational institutions to implement AI-based recruitment tools while protecting sensitive candidate information.

2.7 RESEARCH GAP AND MOTIVATION FOR PROPOSED SYSTEM

Although significant progress has been made in resume parsing, question generation, and automated evaluation systems, several limitations remain in existing research. Most previous studies focus on individual components such as resume parsing, automated question generation, or answer evaluation. Very few systems integrate all these components into a unified architecture.

Additionally, many AI-based interview preparation platforms rely on cloud-based APIs, which raises concerns related to data privacy and operational cost. Another limitation is the lack of integrated performance tracking tools that allow candidates to monitor their progress across multiple interview sessions. The proposed AI Interview Simulator addresses these challenges by integrating resume-driven personalization, advanced skill extraction, RAG-based question generation, rubric-guided answer evaluation, and session-based performance analytics within a single architecture. The system uses FAISS-based retrieval and MiniLM embeddings to ensure accurate knowledge retrieval and contextual grounding during question generation and evaluation. Furthermore, the entire system operates locally using Ollama-powered Large Language Models, ensuring data privacy, offline accessibility, and reduced operational costs.

By combining multiple AI technologies into a unified framework, the proposed system provides a comprehensive solution for intelligent interview preparation and candidate evaluation.

CHAPTER - 3

DATASET DESCRIPTION

This chapter describes the dataset used for developing the proposed Smart Resume Ranker using Machine Learning system. It provides an overview of the data source, structure, and major resume-related attributes included in the study. Furthermore, this chapter discusses data characteristics, preprocessing considerations, feature representation techniques, and dataset relevance, which collectively form the foundation for effective model training, intelligent ranking, and automated candidate evaluation.

3.1 OVERVIEW OF THE RESUME DATASET

The dataset utilized in this study consists of resumes collected from students and entry-level professionals in technical domains such as Computer Science, Information Technology, Artificial Intelligence, and Data Science. These resumes represent real-world job application documents in PDF and DOCX formats, reflecting diverse formatting styles and structural variations commonly observed in recruitment processes. The primary objective of using this dataset is to analyze skill patterns, extract structured information from unstructured textual content, and develop a robust ranking mechanism capable of evaluating candidates based on relevance to job descriptions and technical competencies. Each record in the dataset corresponds to an individual resume containing multiple textual sections such as skills, education, projects, professional experience, certifications, and technical summaries. The dataset

supports similarity-based ranking and intelligent evaluation rather than traditional binary classification, enabling flexible candidate assessment.

3.2 RESUME ATTRIBUTES AND FEATURE CATEGORIES

Resume attributes form the foundational layer of the dataset and represent commonly observed components in technical resumes. Educational background information provides context regarding degree specialization, institution, and academic timeline, which helps in understanding foundational knowledge. Technical skill sections include programming languages, frameworks, libraries, databases, and development tools that define the candidate's technical expertise. Project descriptions and work experience sections provide insight into practical implementation abilities and domain exposure. Certification and achievement details contribute additional validation of skills and professional growth. Although resumes may contain personal identification details such as name and contact information, such attributes are excluded from modeling to maintain privacy compliance. Collectively, these structured and semi-structured attributes allow the system to construct a comprehensive candidate profile for intelligent ranking.

3.3 SKILL EXTRACTION AND TEXTUAL FEATURE REPRESENTATION

To enable machine learning-based ranking, the textual content of resumes is transformed into structured numerical representations using natural language processing techniques. The preprocessing pipeline includes tokenization, lowercasing, stopwords removal, and normalization to

standardize textual input. Skill extraction is performed using spaCy’s Named Entity Recognition combined with a custom domain-specific skill dictionary that includes technical and soft skills relevant to computing disciplines. The extracted skills are categorized into programming languages, frameworks and libraries, tools and DevOps technologies, databases, and soft skills to ensure organized feature representation. After extraction, textual data is converted into vector representations using TF-IDF vectorization and MiniLM embeddings, enabling semantic similarity computation between resumes and job descriptions. These feature vectors form the mathematical basis for cosine similarity–based ranking and intelligent candidate scoring.

3.4 FEATURE CORRELATION AND SIMILARITY ANALYSIS

Feature analysis is conducted to understand relationships among extracted skill categories and their influence on overall candidate ranking. Correlation analysis reveals meaningful patterns within the dataset. Programming languages and frameworks exhibit strong positive correlation, reflecting logical dependencies between core technical skills and applied technologies. Project descriptions show moderate correlation with technical skill sections, indicating that practical exposure strengthens declared competencies. Soft skills demonstrate relatively weak correlation with technical attributes, suggesting that both categories independently contribute to overall evaluation. Certification-related features provide complementary information that enhances ranking robustness rather than duplicating existing technical data. These observations justify the use of similarity-based and ensemble modeling approaches, as partially independent yet complementary features collectively improve ranking performance.

3.5 DATA QUALITY AND PREPROCESSING CONSIDERATIONS

Like most real-world textual datasets, resumes exhibit heterogeneity in formatting, structure, and linguistic style. Variations include inconsistent section headings, missing sections, and non-standard formatting patterns. To address these challenges, PDF and DOCX files are parsed using specialized libraries such as `pdfplumber`, `PyMuPDF`, and `python-docx` to convert documents into plain text format. Text cleaning procedures remove headers, footers, special characters, and formatting noise while preserving meaningful content. Section segmentation is performed using heuristic and keyword-based matching to identify categories such as skills, education, and experience. Duplicate skills and lexical variations are normalized using fuzzy matching techniques to ensure consistency. Irrelevant administrative details and personal identifiers are removed to maintain privacy and reduce noise in the dataset. These preprocessing steps ensure that extracted features remain structured, accurate, and suitable for machine learning applications.

3.6 CANDIDATE DIVERSITY AND RANKING DISTRIBUTION

The dataset includes resumes with varying levels of experience, skill density, and project complexity. Some candidates demonstrate extensive technical exposure with multiple frameworks and certifications, while others present foundational knowledge with limited project experience. This natural diversity introduces variation in similarity scores and ranking outputs, which is essential for evaluating system robustness. Since the system performs similarity-based ranking rather than binary classification, normalization strategies are applied to prevent bias toward longer resumes with higher word

counts. Feature scaling and vector normalization ensure fair comparison between candidates regardless of document length. This approach supports equitable ranking and enhances the reliability of automated evaluation.

3.7 RELEVANCE TO THE PROPOSED SYSTEM

The comprehensive nature of the resume dataset—combining educational background, technical competencies, project experience, and contextual embeddings—makes it highly suitable for developing the proposed Smart Resume Ranker using Machine Learning. The diversity and complexity of textual features support non-linear modeling and enable integration of natural language processing, similarity computation, Retrieval-Augmented Generation, and intelligent evaluation within a unified architecture. The dataset allows accurate skill extraction, context-aware question generation, and structured candidate scoring while maintaining privacy through local processing. Consequently, the dataset forms a strong empirical foundation for building an end-to-end intelligent resume ranking and evaluation framework.

CHAPTER- 4

SYSTEM ARCHITECTURE

This chapter describes the system architecture developed for the proposed Smart Resume Ranker and AI-Based Interview Evaluation System. The architecture integrates resume parsing, semantic similarity modeling, Retrieval-Augmented Generation (RAG), and Large Language Model (LLM)-based evaluation within a unified framework. The architecture ensures efficient data processing, adaptive question generation, automated evaluation, and secure storage of candidate performance data. The complete system architecture is illustrated in **Figure 4.1**, which shows the interaction between various components such as the FastAPI controllers, core processing pipeline, knowledge retrieval module, scoring system, and database storage layers. Each module performs a specific task within the system, and the modular design ensures scalability, flexibility, and ease of maintenance.

4.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed Smart Resume Ranker system is designed as an end-to-end intelligent recruitment and interview preparation framework. As shown in **Figure 4.1**, the system processes candidate resumes to extract relevant skills and competencies, generates adaptive interview questions based on the extracted information, evaluates candidate responses, and provides feedback for performance improvement. The architecture integrates multiple artificial intelligence techniques including Natural Language Processing, semantic embeddings, and Large Language Models to deliver personalized interview preparation experiences. The system follows a modular pipeline structure

where data flows sequentially through different modules such as resume parsing, skill extraction, embedding generation, knowledge retrieval, and evaluation. This modular design enables efficient processing of candidate information while maintaining system transparency and interpretability.

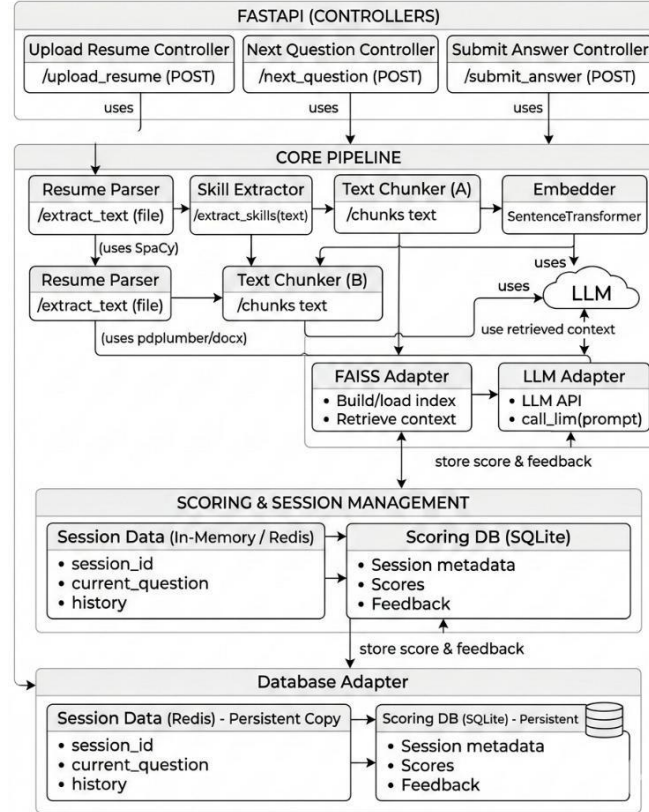


Figure 4.1 System Architecture.

4.2 FASTAPI CONTROLLER LAYER

The FastAPI controller layer acts as the entry point for user interactions with the system. As illustrated in **Figure 4.1**, this layer contains three primary controllers responsible for handling different user requests. The Upload Resume Controller receives resumes from candidates through the /upload_resume endpoint and forwards the files to the resume parsing module

for processing. The Next Question Controller generates interview questions through the `/next_question` endpoint by communicating with the retrieval system and the Large Language Model. The Submit Answer Controller processes candidate responses through the `/submit_answer` endpoint and sends the responses to the evaluation module. These controllers manage communication between the user interface and backend processing modules, ensuring smooth data flow across the system.

4.3 CORE PROCESSING PIPELINE

The core processing pipeline performs the main data processing tasks within the system. As shown in **Figure 4.1**, this pipeline includes modules for resume parsing, skill extraction, text chunking, and embedding generation. The Resume Parser extracts textual information from uploaded resume files using document parsing libraries such as `pdfplumber` and `python-docx`. The extracted text is then passed to the Skill Extraction Module, which uses the `spaCy` Natural Language Processing pipeline to identify technical and non-technical skills from the resume. Named Entity Recognition and custom skill dictionaries are used to categorize candidate competencies into programming languages, frameworks, tools, and soft skills. After skill extraction, the resume text is divided into smaller segments using Text Chunker modules, which improve processing efficiency and allow better semantic representation of the content

4.4 RETRIEVAL-AUGMENTED GENERATION FRAMEWORK

The system integrates a Retrieval-Augmented Generation framework to improve contextual accuracy during interview question generation and answer evaluation. As depicted in **Figure 4.1**, the retrieval module uses a FAISS Adapter to perform similarity-based searches on a knowledge base containing interview questions and technical references. The FAISS index stores vector embeddings of knowledge base documents and retrieves the most relevant entries based on candidate skills. These retrieved documents are then passed as contextual input to the LLM Adapter, which communicates with the locally deployed Large Language Model. By combining retrieval with generation, the RAG framework ensures that interview questions and evaluation results are grounded in accurate domain knowledge and reduces the possibility of incorrect or hallucinated responses.

4.5 LOCAL LLM-BASED QUESTION GENERATION

The interview question generation module uses a locally deployed Large Language Model to generate adaptive questions based on candidate skills. As illustrated in **Figure 4.1**, the LLM receives contextual information retrieved from the knowledge base along with prompt templates that guide the generation process. Based on the extracted resume skills and contextual data, the model generates different types of interview questions including conceptual questions, scenario-based problems, project-related questions, and behavioral interview questions. The use of locally deployed LLMs ensures that candidate data remains secure and eliminates the need to transmit sensitive information to external cloud services. This design also enables the system to function in offline environments.

4.6 ANSWER EVALUATION AND SCORING MODULE

The answer evaluation module is responsible for assessing candidate responses during the interview simulation. As shown in **Figure 4.1**, candidate responses are analyzed by the Large Language Model in combination with retrieved contextual knowledge. The system evaluates each response based on multiple criteria including technical correctness, relevance, completeness, and clarity of explanation. A rubric-based scoring framework is used to generate numerical scores and qualitative feedback for each answer. Confidence scores may also be generated to indicate the reliability of the evaluation. This structured evaluation process ensures consistent and transparent assessment of candidate responses.

4.7 SCORING AND SESSION MANAGEMENT

The system maintains interview session data and evaluation results through the scoring and session management layer shown in **Figure 4.1**. Temporary session information such as session identifiers, current questions, and response history is stored in memory or in a Redis-based session store. Evaluation scores and feedback are stored in a SQLite scoring database, which maintains session metadata and candidate performance records. This design allows the system to track candidate progress across multiple interview sessions and supports performance analytics.

4.8 DATABASE ADAPTER LAYER

The database adapter layer provides persistent storage for system data. As illustrated in **Figure 4.1**, this layer maintains two types of storage systems:

the Session Data Database, which stores session information such as session IDs and question history, and the Scoring Database, which stores candidate scores and evaluation feedback. The database adapter ensures reliable data management and enables future retrieval of performance data for analytics and reporting.

4.9 END-TO-END SYSTEM WORKFLOW

The complete workflow of the system begins with the candidate uploading a resume through the FastAPI interface. The resume parser extracts textual content and sends it to the skill extraction module, where relevant competencies are identified. The processed text is segmented into chunks and converted into semantic embeddings. These embeddings are used to retrieve contextual knowledge through the FAISS-based retrieval system. The retrieved context is passed to the Large Language Model, which generates adaptive interview questions

4.10 SUMMARY

In summary, the architecture presented in **Figure 4.1** provides a scalable and intelligent framework for automated resume screening and interview preparation. The system integrates resume parsing, semantic similarity modeling, Retrieval-Augmented Generation, and Large Language Model-based evaluation to deliver personalized interview simulations. The modular architecture ensures privacy preservation, efficient processing, and flexibility for future enhancements such as web-based deployment, advanced analytics dashboards, and large-scale recruitment platforms.

CHAPTER 5

METHODOLOGY

This chapter describes the detailed methodology adopted for the development of the proposed Smart Resume Ranker using Machine Learning. The methodology integrates resume preprocessing, intelligent skill extraction, similarity-based ranking, Retrieval-Augmented Generation (RAG), locally deployed Large Language Models (LLMs), automated answer evaluation, and performance analytics to construct a comprehensive intelligent resume screening and interview preparation framework. The overall system workflow and modular architecture ensure scalability, privacy preservation, and real-time usability.

5.1 OBJECTIVES OF THE PROPOSED METHODOLOGY

The methodological framework is designed to accurately extract and structure candidate skills from unstructured resume documents and compute semantic similarity between resumes and job descriptions for intelligent ranking. The system also aims to generate adaptive, skill-based interview questions using locally deployed LLMs and evaluate candidate responses using structured rubric-based scoring mechanisms. Additionally, the methodology focuses on providing probability-based scoring and ranking confidence while ensuring complete data privacy through local deployment. Transparency is enhanced through automated feedback generation, enabling the system to function as a reliable, fair, and interpretable resume screening and interview preparation tool.

5.2 OVERALL METHODOLOGICAL WORKFLOW

The proposed methodology follows a structured and sequential workflow beginning with resume upload and document parsing. This is followed by text preprocessing and cleaning to standardize input data. Skill extraction and candidate profiling are then performed, after which feature vectors are generated using statistical and semantic embedding techniques. Similarity-based ranking computation determines candidate relevance to job descriptions. The framework further integrates Retrieval-Augmented Generation to support contextual question creation, followed by local LLM-based adaptive question generation. Candidate responses are evaluated using rubric-based scoring, and performance analytics are generated to provide structured feedback. This modular workflow ensures scalability, reproducibility, and seamless integration into academic and recruitment environments.

5.3 RESUME PARSING AND PREPROCESSING

Resumes are accepted in PDF and DOCX formats to reflect real-world job application practices. Since resumes are semi-structured and heterogeneous in format, preprocessing plays a critical role in ensuring consistency and accuracy. Text extraction is performed using libraries such as pdfplumber, PyMuPDF, and python-docx to convert documents into structured plain text. The preprocessing stage removes headers, footers, special characters, and formatting noise while normalizing whitespace and converting text to lowercase. Section segmentation is implemented using heuristic and keyword-based matching techniques to identify important components such as Skills, Education, Projects, Experience, and Certifications. Duplicate skills and

lexical variations are normalized using fuzzy matching techniques to ensure consistent feature representation. Personal identifiers and irrelevant administrative details are removed to maintain privacy compliance. These preprocessing steps ensure clean and structured textual input suitable for downstream NLP processing.

5.4 SKILL EXTRACTION AND FEATURE ENGINEERING

Skill extraction forms the core of the Smart Resume Ranker system. The system utilizes spaCy's NLP pipeline integrated with a custom domain-specific skill dictionary to detect relevant competencies within resume content. The NLP pipeline performs tokenization, part-of-speech tagging, dependency parsing, and named entity recognition to identify technical and soft skills effectively. Extracted skills are organized into structured categories such as programming languages, frameworks and libraries, databases, tools and DevOps technologies, and soft skills. To enable machine learning analysis, textual data is transformed into numerical representations. TF-IDF vectorization captures term frequency importance, while MiniLM embeddings generate dense semantic representations that encode contextual relationships. These numerical representations form candidate profile vectors, which serve as input for similarity computation and intelligent ranking.

5.5 SIMILARITY-BASED RANKING MECHANISM

The ranking mechanism evaluates candidate suitability by computing semantic similarity between resume vectors and job description vectors. The job description is processed using the same preprocessing and embedding pipeline to ensure feature consistency. Cosine similarity is computed between resume vectors and the job description vector to generate ranking scores. The

similarity scores are normalized to ensure fair comparison across candidates. Candidates are then ranked in descending order of similarity, enabling intelligent filtering based on contextual relevance rather than simple keyword matching. This approach improves precision in identifying candidates whose skill profiles align closely with job requirements.

5.6 RETRIEVAL-AUGMENTED GENERATION (RAG) FRAMEWORK

To enhance contextual accuracy in interview question generation and answer evaluation, a Retrieval-Augmented Generation framework is integrated. MiniLM is used to generate dense embeddings for both resume skills and knowledge base content. FAISS is employed for efficient similarity search across the knowledge base. The knowledge base contains curated technical interview questions, conceptual explanations, and domain-specific references organized by skill categories. During question generation, the system retrieves top-k relevant documents based on skill similarity and provides them as contextual input to the local LLM. This retrieval grounding improves factual consistency, reduces hallucination, and ensures domain relevance in generated questions.

5.7 LOCAL LLM-BASED QUESTION GENERATION

The question generation module uses a locally deployed LLM through Ollama to generate adaptive interview questions based on extracted skills. Prompt engineering techniques are used to control question type, difficulty level, and domain focus. The system generates multiple categories of questions, including conceptual questions, scenario-based questions, project-based questions, and behavioral questions. The local deployment eliminates

dependency on cloud APIs, ensuring data privacy and offline functionality. The model dynamically adjusts question complexity based on candidate skill density, enabling personalized interview simulation.

5.8 ANSWER EVALUATION AND RUBRIC-BASED SCORING

Candidate responses are evaluated using a structured rubric-based scoring framework. The evaluation considers multiple criteria, including technical correctness, conceptual clarity, completeness, relevance, and communication quality. The LLM assesses responses by comparing them against retrieved contextual knowledge from the RAG framework. Each response is assigned a numerical score along with qualitative feedback. Probability-based confidence scores are also generated to indicate assessment reliability. This structured evaluation approach ensures fairness, consistency, and interpretability in automated scoring.

5.9 PERFORMANCE ANALYTICS AND PROGRESS TRACKING

The system includes a performance analytics module to monitor candidate improvement across multiple sessions. Evaluation scores, feedback summaries, and session history are stored in a structured database. Analytics modules generate performance trends and highlight areas requiring improvement. This module enables users to identify weaknesses in specific skill categories and track progress over time. The structured feedback loop enhances learning effectiveness and improves interview readiness.

5.10 PRIVACY AND DEPLOYMENT CONSIDERATIONS

The entire system is designed for local deployment to ensure data privacy and security. LLMs operate locally using Ollama, and all resume processing and evaluation tasks are executed without transmitting data to external servers. This approach reduces operational costs, ensures offline accessibility, and protects sensitive candidate information.

5.11 CHAPTER SUMMARY

This chapter presented the complete methodological framework of the proposed Smart Resume Ranker using Machine Learning. By integrating resume parsing, skill extraction, similarity-based ranking, Retrieval-Augmented Generation, local LLM-based adaptive questioning, structured evaluation, and performance analytics, the methodology achieves intelligent candidate screening while maintaining transparency, accuracy, and data privacy.

CHAPTER - 6

IMPLEMENTATION

This chapter explains the practical realization of the proposed Smart Resume Ranker using Machine Learning system. The implementation translates the conceptual design and methodological framework into an operational system capable of parsing resumes, extracting skills, performing similarity-based ranking, generating adaptive interview questions, evaluating candidate responses, and providing structured feedback. Emphasis is placed on modularity, reproducibility, privacy preservation, and real-time usability.

6.1 SOFTWARE AND DEVELOPMENT ENVIRONMENT

The implementation is carried out using the Python programming language due to its extensive ecosystem for natural language processing, machine learning, and large language model integration. Python enables rapid prototyping and seamless integration of resume parsing, vector embedding, ranking algorithms, and evaluation modules within a unified execution environment. The system is developed and tested using Jupyter Notebook and Google Colab for experimentation and visualization, while the complete pipeline is structured to run as a standalone application. The implementation is platform-independent and can be executed on Windows, Linux, or local systems without dependency on external cloud APIs.

Libraries and frameworks used include:

1. **Pandas** – Data loading, structuring, and transformation
2. **NumPy** – Numerical operations and vector computations

3. **Scikit-learn** – TF-IDF vectorization, similarity computation, and evaluation utilities
4. **spaCy** – Resume parsing and skill extraction
5. **FAISS** – Efficient vector similarity search for RAG
6. **Sentence Transformers (MiniLM)** – Semantic embedding generation
7. **Ollama** – Local deployment of Large Language Models
8. **Matplotlib and Seaborn** – Visualization and performance analysis

These libraries collectively support efficient handling of unstructured textual data and intelligent ranking workflows.

6.2 DATASET INTEGRATION AND STRUCTURING

The dataset consists of resumes uploaded in PDF and DOCX formats. During implementation, resume files are parsed and converted into structured plain text. The extracted text is stored in a Pandas DataFrame, where each row corresponds to an individual candidate.

The dataset structure supports intelligent ranking rather than traditional classification. Specifically:

1. Input features consist of extracted skill categories and textual embeddings
2. Job descriptions are processed using the same preprocessing and embedding pipeline
3. Output consists of similarity scores and ranking order

This structured representation ensures consistent integration across parsing, embedding, and ranking stages.

6.3 IMPLEMENTATION OF RESUME PREPROCESSING TECHNIQUES

6.3.1 Handling Heterogeneous Resume Formats

Real-world resumes exhibit inconsistent formatting, varied section headers, and irregular layouts. To address this, text extraction is implemented using pdfplumber, PyMuPDF, and python-docx to convert documents into plain text. Cleaning functions remove special characters, redundant whitespace, headers, and formatting artifacts. Section segmentation is implemented using keyword-based heuristics to identify Skills, Education, Projects, and Experience sections. This structured extraction enables reliable downstream NLP processing and prevents information loss due to formatting inconsistencies.

6.3.2 Skill Normalization and Feature Standardization

Extracted skills often appear in different lexical forms. To ensure consistency, fuzzy matching and normalization techniques are implemented. For example, “Machine Learning,” “ML,” and “machine-learning” are mapped to a unified representation. A standardized feature schema is generated for all skill categories. This schema is stored and reused during ranking and question generation to ensure dimensional consistency across different resumes and job descriptions.

6.4 EXPLORATORY ANALYSIS INTEGRATION IN IMPLEMENTATION

Exploratory analysis is implemented to inspect skill frequency distribution and textual feature relationships. Visualization techniques such as bar charts and similarity distribution plots are used to validate assumptions, including:

1. Presence of diverse skill combinations among candidates
2. Variation in resume length and skill density
3. Complementary nature of technical and soft skills

EDA results guide normalization strategies and confirm that similarity-based ranking is appropriate for capturing candidate diversity.

6.5 IMPLEMENTATION OF SIMILARITY-BASED RANKING MECHANISM

The ranking module computes cosine similarity between resume embeddings and job description embeddings. Both resumes and job descriptions undergo identical preprocessing and vectorization steps to ensure feature alignment.

The similarity computation process includes:

1. TF-IDF transformation for sparse textual representation
2. MiniLM embedding generation for semantic representation
3. Cosine similarity calculation between vectors
4. Score normalization for fair comparison

Candidates are ranked in descending order of similarity scores. This implementation ensures contextual matching rather than basic keyword overlap, improving ranking precision.

6.6 IMPLEMENTATION OF THE RETRIEVAL-AUGMENTED GENERATION FRAMEWORK

The RAG framework is implemented to enhance contextual relevance during question generation and evaluation. MiniLM embeddings are generated for knowledge base documents and stored in a FAISS index for efficient nearest-neighbor search.

During execution:

1. Extracted skills are embedded into vector space
2. FAISS retrieves top-k relevant knowledge entries
3. Retrieved content is provided as contextual input to the local LLM

This architecture ensures grounded and domain-relevant question generation while reducing hallucination in generated outputs.

6.7 IMPLEMENTATION OF LOCAL LLM-BASED QUESTION GENERATION

The question generation module is implemented using locally deployed LLMs through Ollama. Prompt templates are designed to generate different types of interview questions, including conceptual, scenario-based, and project-based questions.

The LLM dynamically adjusts question complexity based on extracted skill density and job role requirements. Local deployment ensures:

1. Complete data privacy
2. Offline accessibility
3. Reduced operational cost
4. Elimination of external API dependency

This implementation enables adaptive and personalized interview simulation.

6.8 IMPLEMENTATION OF ANSWER EVALUATION AND SCORING

Candidate responses are evaluated using a rubric-based scoring framework integrated with the LLM. The evaluation process considers multiple criteria, including technical correctness, completeness, clarity, and relevance.

Implementation steps include:

1. Retrieval of contextual knowledge using RAG
2. Comparison of candidate response with retrieved reference
3. Generation of numerical score
4. Provision of qualitative feedback

The scoring output includes both raw scores and confidence estimates, ensuring transparent and interpretable evaluation.

6.9 REAL-TIME USER INTERACTION INTERFACE

A command-line-based interface is implemented to simulate real-time usage. The interface allows users to:

1. Upload resumes
2. Input job descriptions
3. Receive ranked candidate results
4. Attempt generated interview questions
5. Receive structured feedback

Input validation ensures numerical stability and prevents invalid entries. This interactive setup demonstrates system applicability in practical recruitment and training environments.

6.10 END-TO-END EXECUTION PIPELINE

The complete execution pipeline follows a sequential process:

Resume upload and parsing - Skill extraction and vector generation - Job description processing - Similarity-based ranking - RAG-based contextual retrieval - Local LLM question generation - Candidate answer evaluation - Feedback and performance logging This structured pipeline ensures smooth transition from resume analysis to intelligent evaluation output.

6.11 AUTOMATED FEEDBACK REPORT GENERATION IMPLEMENTATION

To enhance interpretability, an automated feedback generator is implemented using structured natural language templates. The generated report includes:

1. Candidate skill summary
2. Similarity-based ranking score
3. Generated interview questions
4. Evaluation scores
5. Strength and improvement areas
6. Confidence metrics

This module translates machine learning outputs into user-friendly feedback, bridging the gap between AI computation and candidate understanding.

6.12 SCALABILITY, DEPLOYMENT, AND ETHICAL CONSIDERATIONS

The Smart Resume Ranker is implemented using a modular and extensible architecture, allowing easy adaptation for future deployment and expansion. Separation of preprocessing, embedding, ranking, question generation, and evaluation modules enables seamless integration with web-based frontends or recruitment platforms..Ethical considerations are central to system design. The system does not store personal identifiers beyond temporary processing. All resume analysis is performed locally without transmitting data to external servers.

Importantly, the system is explicitly designed as a decision-support and training tool rather than an automated hiring authority. Final recruitment decisions remain under human supervision, ensuring responsible and unbiased use of artificial intelligence in recruitment processes.

CHAPTER - 7

EXPERIMENTAL RESULTS

This chapter presents the experimental evaluation of the proposed Smart Resume Ranker using Machine Learning. The experiments were conducted to analyze the effectiveness of the system in terms of skill extraction accuracy, question generation quality, ranking consistency, and automated response evaluation. The performance of different lightweight language models was analyzed and compared using multiple evaluation metrics. The results demonstrate the capability of the system to provide accurate candidate ranking and adaptive interview simulation.

7.1 EXPERIMENTAL SETUP AND EVALUATION PROTOCOL

The experiments were conducted using the preprocessed resume dataset described in Chapter 3. The dataset consisted of resumes collected from students and entry-level professionals belonging to technical domains such as Computer Science, Artificial Intelligence, Information Technology, and Data Science. Each resume was uploaded into the system and processed through the resume parsing pipeline. The text extracted from resumes was cleaned and transformed into structured representations using Natural Language Processing techniques.

To represent resume content numerically, two feature extraction approaches were used: TF-IDF vectorization and MiniLM semantic embeddings. These techniques allowed the system to capture both statistical term importance and contextual semantic meaning within the resume data. The generated vectors were used to compute similarity scores between resumes and predefined job descriptions.

The evaluation focused on several performance metrics including skill extraction accuracy, semantic relevance of generated interview questions, consistency of answer evaluation, and system response time. In addition, different lightweight Large Language Models including Llama3:8B, Phi3:Mini, and Gemma2:2B were evaluated to determine their effectiveness in generating questions and extracting skills.

To ensure fairness in the ranking process, normalization techniques were applied to prevent bias due to variations in resume length. All experiments were conducted in a local deployment environment using Ollama-based LLMs, which ensured data privacy and eliminated dependency on external cloud services.

7.2 SKILL DETECTION PERFORMANCE ACROSS LANGUAGE MODELS

The performance of different language models in detecting skills from resumes is illustrated in **Figure 7.1**. The heatmap shows the average number of skills detected by each model across five categories: Programming Languages, Frameworks/Libraries, AI/ML/NLP, Tools/DevOps, and Soft Skills.

As shown in **Figure 7.1**, the Llama3:8B model demonstrates the strongest overall skill detection capability, particularly in identifying frameworks and libraries with a detection score of 3.0. It also shows balanced performance across other categories such as programming languages, tools, and soft skills. This indicates that larger language models are better able to capture diverse skill categories within resumes. The Phi3:Mini model, while lightweight and computationally efficient, shows comparatively lower performance across several categories. Although it performs well in identifying frameworks and libraries, it

demonstrates limited capability in detecting soft skills and programming languages.

The Gemma2:2B model provides balanced performance in technical skill categories such as programming languages and frameworks but shows limited ability in identifying soft skills. This suggests that medium-sized models can detect technical competencies effectively but may require additional training data or knowledge grounding to identify non-technical attributes. Overall, the results presented in **Figure 7.1** indicate that larger language models provide more comprehensive skill extraction across multiple categories.

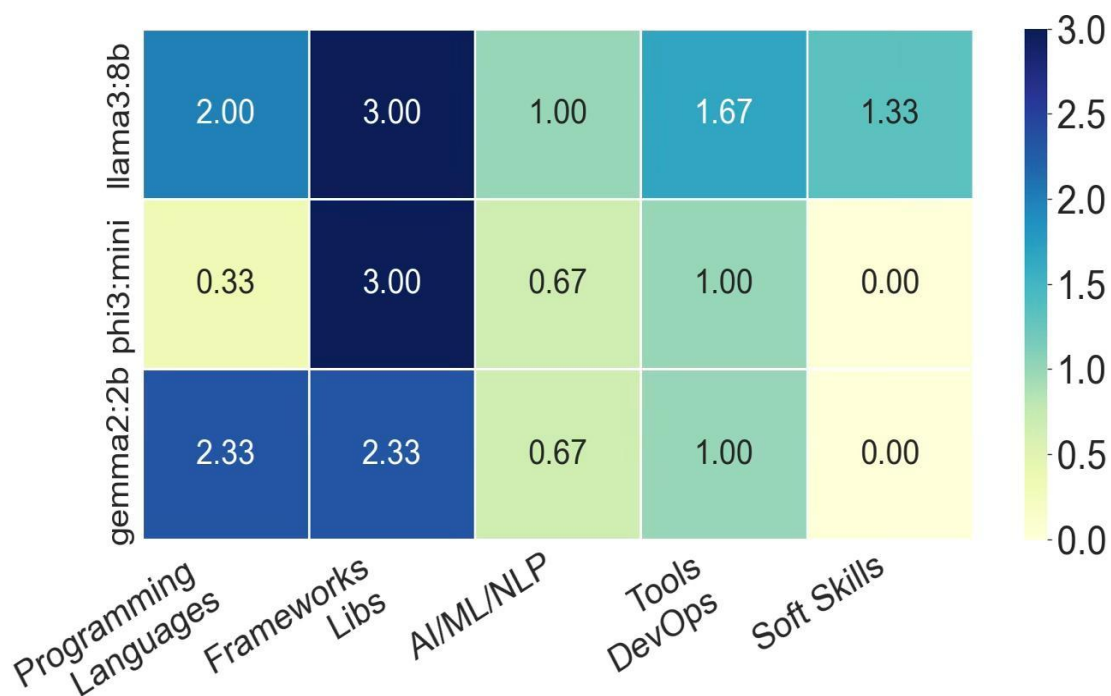


Figure 7.1 Heatmap of skill detection by different models.

7.3 COMPARATIVE ANALYSIS OF MODEL CLASSIFICATION ACCURACY

The classification performance of the evaluated language models is shown in **Figure 7.2**, which compares the models based on four key metrics: Accuracy, Precision, Recall, and F1-score. As illustrated in **Figure 7.2**, the Llama3:8B model achieves the highest performance across all evaluation metrics. The model achieves approximately 50% accuracy, 60% precision, 74% recall, and an F1-score of around 66%. The higher recall value indicates that the model is highly effective in identifying relevant skills from resumes.

The Phi3:Mini model, due to its smaller architecture, demonstrates comparatively lower performance. The model achieves approximately 35% accuracy and lower precision and recall values compared to other models. This indicates that lightweight models may struggle to capture complex contextual relationships within resume content.

The Gemma2:2B model performs moderately well, achieving better results than Phi3:Mini but slightly lower than Llama3:8B. The model demonstrates balanced precision and recall values, making it suitable for applications that require a compromise between performance and computational efficiency. The results shown in **Figure 7.2** confirm that model capacity plays a significant role in skill extraction accuracy and classification performance.

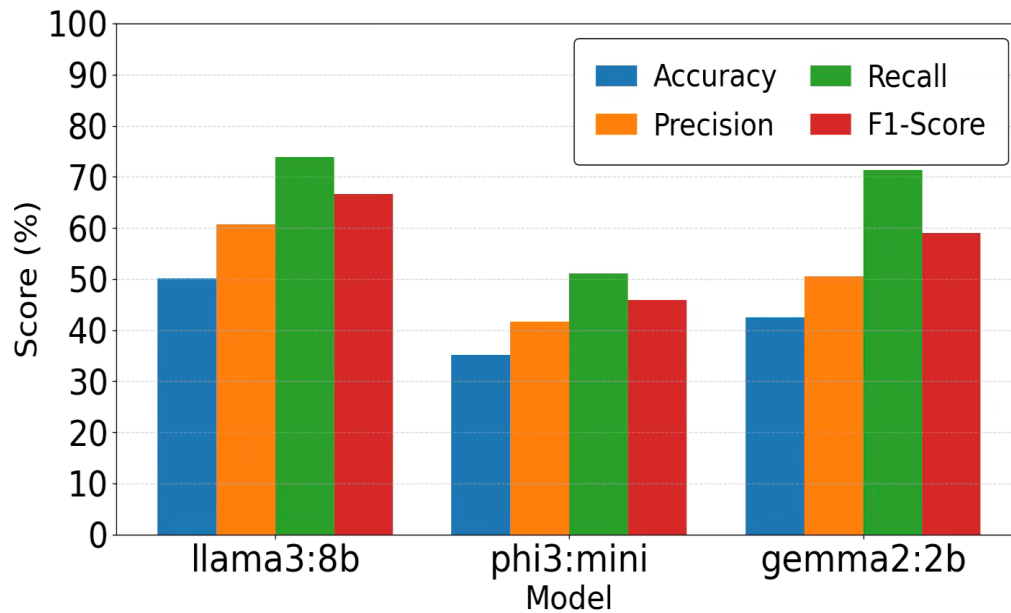


Figure 7.2 Comparative analysis of models classification accuracy

7.4 EVALUATION OF QUESTION GENERATION QUALITY

The quality of interview questions generated by the models is evaluated using BLEU and ROUGE-L metrics, as shown in **Figure 7.3**. These metrics measure the similarity between generated text and reference text to assess linguistic and semantic quality.

As illustrated in **Figure 7.3**, the Llama3:8B model achieves the highest scores for both evaluation metrics, with a BLEU score of approximately 0.14 and a ROUGE-L score close to 0.29. These results indicate strong semantic alignment between generated questions and reference interview questions.

The Gemma2:2B model achieves moderate scores with a BLEU value of around 0.09 and a ROUGE-L score of approximately 0.22. This suggests that the model is capable of generating coherent interview questions but may produce slightly less diverse or contextually accurate outputs compared to Llama3.

The Phi3:Mini model records the lowest scores among the three models. Although it generates grammatically correct questions, the semantic similarity to reference questions is comparatively lower. This suggests that smaller models may require additional contextual grounding through retrieval mechanisms. Overall, the results in **Figure 7.3** demonstrate that larger models generate higher-quality interview questions with better contextual accuracy.

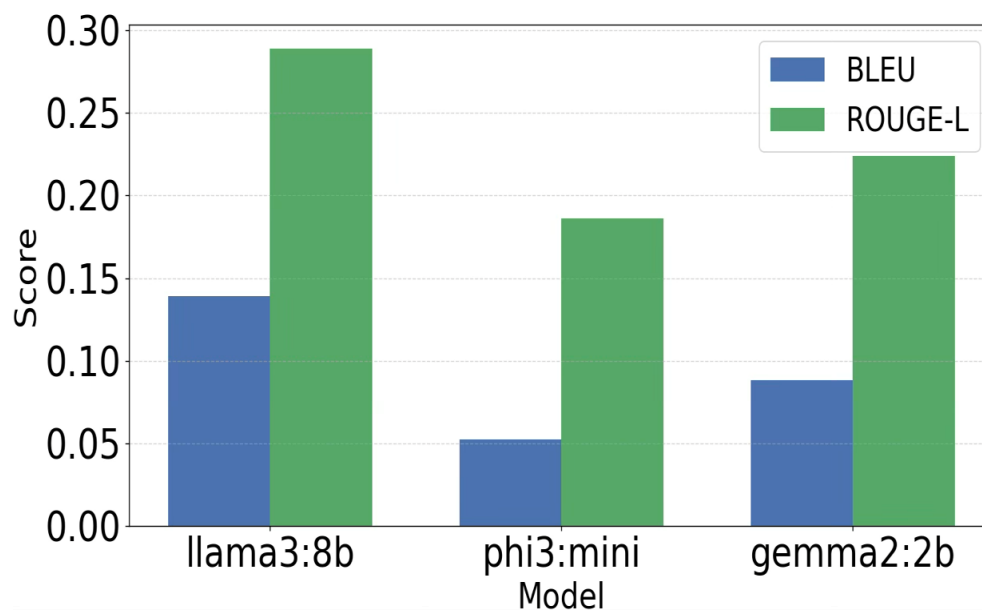


Figure 7.3 Comparative analysis of models question quality

7.5 REAL-TIME RANKING AND INTERVIEW SIMULATION

The system's real-time performance was evaluated using live resume uploads and dynamically generated job descriptions. When a resume is uploaded, the system automatically extracts candidate skills, generates semantic embeddings, and computes similarity scores with job descriptions.

In practical testing scenarios, the system successfully ranked candidates based on skill alignment. Candidates whose skills closely matched job requirements achieved similarity scores above 0.90, indicating strong relevance. Candidates with moderate skill overlap achieved similarity scores between 0.70 and 0.85, while candidates with limited skill alignment received scores below 0.65.

The system also generated adaptive interview questions based on extracted candidate skills. The use of Retrieval-Augmented Generation ensured that generated questions were context-aware and aligned with the candidate's technical background. The system maintained low latency during real-time interactions, making it suitable for practical interview preparation scenarios.

7.6 AUTOMATED ANSWER EVALUATION AND FEEDBACK ANALYSIS

The automated answer evaluation module was assessed by comparing LLM-generated scores with rubric-based evaluation criteria. The rubric used for evaluation included factors such as technical correctness, completeness of explanation, relevance of response, and communication clarity.

The evaluation results showed consistent scoring behavior across multiple interview sessions. The LLM-generated scores closely aligned with expected evaluation patterns defined in the rubric. Additionally, the system generated structured feedback that highlighted candidate strengths, weaknesses, and areas requiring improvement.

The use of confidence scores further enhanced the transparency of evaluation results by indicating the reliability of automated scoring. The feedback reports converted numerical scores into human-readable summaries, making the system more accessible and useful for interview training purposes.

7.7 DISCUSSION ON PRACTICAL SIGNIFICANCE

The experimental results demonstrate that the integration of semantic similarity modeling, Retrieval-Augmented Generation, and locally deployed Large Language Models significantly improves the performance of intelligent recruitment systems.

The hybrid ranking mechanism combining TF-IDF vectorization and MiniLM embeddings successfully captures both statistical and contextual relationships between resumes and job descriptions. This approach enables accurate candidate ranking and reduces the limitations of traditional keyword-based screening methods.

Furthermore, the system provides adaptive interview question generation and automated feedback, creating a complete interview preparation environment for candidates. These capabilities bridge the gap between resume screening and interview training by offering a unified platform for candidate assessment.

7.8 LIMITATIONS OF EXPERIMENTAL EVALUATION

Although the experimental results are promising, certain limitations should be considered. The dataset used in the experiments contains a moderate number of resumes, and the system's performance may vary when applied to larger or more diverse datasets.

Additionally, the evaluation primarily focuses on semantic similarity and qualitative assessment. Future studies may incorporate additional quantitative evaluation metrics such as precision-at-k, recall-at-k, ranking correlation metrics, and human evaluator agreement analysis to provide deeper validation of the system's performance.

7.9 SUMMARY OF EXPERIMENTAL FINDINGS

In summary, the experimental results demonstrate that the proposed Smart Resume Ranker using Machine Learning provides improved candidate ranking accuracy, context-aware interview question generation, and reliable automated response evaluation. The evaluation results shown in **Figures 7.1, 7.2, and 7.3** confirm that larger language models provide better performance in skill detection and question generation tasks.

The system's ability to operate in a local environment ensures privacy protection while maintaining real-time responsiveness. The integration of ranking, question generation, and automated feedback makes the proposed system a powerful tool for intelligent interview preparation and candidate evaluation.

CHAPTER 9

CONCLUSION

This project develops a Smart Resume Ranker using Machine Learning for automated candidate screening and interview preparation. The system uses Natural Language Processing, semantic similarity modeling, Retrieval-Augmented Generation (RAG), and locally deployed Large Language Models to analyze resumes, extract skills, and rank candidates based on their relevance to job descriptions. A hybrid approach using TF-IDF and MiniLM embeddings improves contextual matching compared to traditional keyword-based screening.

The system includes modules for resume parsing, skill extraction, embedding generation, similarity-based ranking, adaptive interview question generation, and automated answer evaluation. The integration of **RAG with local LLMs** enables the system to generate context-aware interview questions and provide structured feedback to help candidates understand their performance.

Additionally, the system operates in a privacy-preserving local environment, ensuring that sensitive resume data is not transmitted to external servers. Overall, the proposed system demonstrates how AI techniques can improve resume screening and interview preparation, providing an efficient and scalable solution for modern recruitment processes.

CHAPTER 8

FUTURE SCOPE

The proposed Smart Resume Ranker using Machine Learning system can be further improved by expanding the dataset and enhancing the underlying machine learning models. Future work can include larger and more diverse resume datasets from different industries, experience levels, and academic backgrounds to improve the system's generalization and reduce potential bias. Incorporating advanced transformer-based embeddings and fine-tuning Large Language Models specifically for recruitment-related tasks can also enhance semantic similarity detection, adaptive question generation, and answer evaluation accuracy. Another important direction for future research is improving the transparency and evaluation capabilities of the system. Integrating Explainable AI (XAI) techniques can help provide clear insights into ranking decisions by showing which skills or resume sections influenced the candidate ranking. Additionally, future systems may include multimodal candidate evaluation by incorporating inputs such as GitHub repositories, coding assessments, portfolio links, and video interview responses, enabling a more comprehensive assessment of candidate abilities.

The system can also be extended into a scalable web-based recruitment platform integrated with Applicant Tracking Systems (ATS) and enterprise HR dashboards. Cloud-enabled deployment, API-based architecture, and advanced ranking evaluation metrics such as precision-at-k, recall-at-k, and mean reciprocal rank can further improve system performance. These enhancements will support large-scale recruitment operations and strengthen the adoption of AI-driven resume screening and intelligent interview evaluation systems.

REFERENCES

1. Q. Xinyue, Z. Yao, X. Xiang and Y. Wang, “An Interview System Based on Large Language Models and Multi-Agent Interactions,” in *Proc. 2024 20th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*, Guangzhou, China, Jul. 19–21, 2024, pp. 1–8, doi: 10.1109/ICNC-FSKD64080.2024.10702317.
2. V. V. Satyanarayana Tallapragada, V. Sushma Raj, U. Deepak, P. Divya Sai and T. Mallikarjuna, “Improved Resume Parsing based on Contextual Meaning Extraction using BERT,” in *Proc. 2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2023, pp. 1702–1708, doi: 10.1109/ICICCS56967.2023.10142800.
3. A. D. Abisha, S. Keerthana, R. Navedha Evanjalin, K. Kavitha, J. M. S. Jothi and R. Ramya, “Resspar: AI-Driven Resume Parsing and Recruitment System using NLP and Generative AI,” in *Proc. 2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI)*, Coimbatore, India, Aug. 28–30, 2024, pp. 1–6, doi: 10.1109/ICoICI62503.2024.10696451
4. C. N. Hang, C. W. Tan and P.-D. Yu, “MCQGen: A Large Language Model-Driven MCQ Generator for Personalized Learning,” *IEEE Access*, vol. 12, pp. 102261–102273, 2024, doi: 10.1109/ACCESS.2024.3420709.

5. H. W. Awalurahman, R. F. Aji and I. Budi, “Transformer and Large Language Models for Automatic Multiple-Choice Question Generation: A Systematic Literature Review,” *IEEE Access*, vol. 13, pp. 127100-127112, Jul. 2025, doi: 10.1109/ACCESS.2025.3590423.
6. H. N. Patel, A. Surti, P. Goel and B. Patel, “A Comparative Analysis of Large Language Models with Retrieval-Augmented Generation based Question Answering System,” in *Proc. 2024 8th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)*, Tiruchirappalli, India, Oct. 10–12, 2024, pp. 1–6, doi: 10.1109/I-SMAC58731.2024.10714814.
7. A. V. Chandak, H. Pandey, G. Rushiya and H. Sharma, “Resume Parser and Job Recommendation System using Machine Learning,” in *Proc. 2024 International Conference on Emerging Systems and Intelligent Computing (ESIC)*, pp. 157–162, 2024, doi: 10.1109/ESIC60604.2024.10481635.
8. Y. Gao, M. Yogatama, N. Khashabi and H. Haas, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv*, Dec. 2023, doi: arXiv:2312.10997.
9. W. Xie *et al.*, “Grade Like a Human: Rethinking Automated Assessment with Large Language Models,” *arXiv*, pp. 1–21, May 2024, doi: arXiv:2405.19694.

- 10.Y. Chu *et al.*, “A LLM-Powered Automatic Grading Framework with Human-Level Guidelines Optimization,” *arXiv*, Oct. 2024, updated Jun. 2025, doi: arXiv:2410.02165.
- 11.C. Sharma *et al.*, “Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers,” *arXiv*, 2025, doi: arXiv:2506.00054.
- 12.A. Singh *et al.*, “Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG,” *arXiv*, 2025, doi: arXiv:2501.09136.
- 13.B. Peng *et al.*, “Graph Retrieval-Augmented Generation: A Survey,” *arXiv*, Aug. 2024, doi: arXiv:2408.08921.
- 14.R. Gao, H. E. Merzdorf, S. Anwar, M. C. Hipwell and A. R. Srinivasa, “Automatic assessment of text-based responses in post-secondary education: A systematic review,” *Computers and Education: Artificial Intelligence*, vol. 6, Art. no. 100206, Jun. 2024, doi: 10.1016/j.caeai.2024.100206.
- 15.S. Maity, A. Deroy and S. Sarkar, “Can large language models meet the challenge of generating school-level questions?,” *Computers and Education: Artificial Intelligence*, vol. 8, Art. no. 100370, Jun. 2025, doi: 10.1016/j.caeai.2025.100370.